

hazard_detection_unit.sv

```
module hazard_detection_unit(
    input Memread,
    input [31:0] inst,
    input [4:0] Rd,
    output reg stall
);

    initial begin
        stall = 1'b0;
    end

    always @(*) begin
        if (Memread == 1'b1 && ((Rd == inst[19:15]) || (Rd == inst[24:20])))
            stall = 1'b1;
        else
            stall = 1'b0;
        end
    end

endmodule
```

reg_execute_memory.sv

```
module reg_execute_memory(
    input clk,
    input reset,
    input [63:0] Adder_out,      //adder output
    input [63:0] Result_in_alu, //64bit alu output
    input Zero_in,              //64bit alu output
    input [63:0] writedata_in,  //2 bit mux2by1 output
    input [4:0] Rd_in,          //IDEX output
    input branch_in,
    input memread_in,
    input memtoreg_in,
    input memwrite_in,
    input regwrite_in,          //IDEXX outputs
    input flush,
    input addermuxselect_in,
    output reg [63:0] Adderout,
    output reg zero,
    output reg [63:0] result_out_alu,
    output reg [63:0] writedata_out,
    output reg [4:0]rd,
    output reg Branch,
    output reg Memread,
    output reg Memtoreg,
    output reg Memwrite,
    output reg Regwrite,
    output reg addermuxselect
);
```

```

always @ (posedge clk) begin
    if (reset == 1'b1 || flush == 1'b1) begin
        Adderout = 64'b0;
        zero = 1'b0;
        result_out_alu = 63'b0;
        writedata_out = 64'b0;
        rd = 5'b0;
        Branch = 1'b0;
        Memread = 1'b0;
        Memtoreg = 1'b0;
        Memwrite = 1'b0;
        Regwrite = 1'b0;
        addermuxselect = 1'b0;
    end
    else begin
        Adderout = Adder_out;
        zero = Zero_in;
        result_out_alu = Result_in_alu;
        writedata_out = writedata_in;
        rd = Rd_in;
        Branch = branch_in;
        Memread = memread_in;
        Memtoreg = memtoreg_in;
        Memwrite = memwrite_in;
        Regwrite = regwrite_in;
        addermuxselect = addermuxselect_in;
    end
end
endmodule

```

control_unit.sv

```

module control_unit(
    input [6:0] opcode,
    input stall,

    output reg branch,
    output reg memread,
    output reg memtoreg,
    output reg memwrite,
    output reg aluSrc,
    output reg regwrite,
    output reg [1:0] Aluop
);

always @(*) begin
    if (opcode == 7'b0000011) begin
        aluSrc = 1'b1;
    end
end

```

```

    memtoreg = 1'b1;
    regwrite = 1'b1;
    memread = 1'b1;
    memwrite = 1'b0;
    branch = 1'b0;
    Aluop = 2'b00;
end
else if (opcode == 7'b0100011) begin
    aluSrc = 1'b1;
    memtoreg = 1'bx;
    regwrite = 1'b0;
    memread = 1'b0;
    memwrite = 1'b1;
    branch = 1'b0;
    Aluop = 2'b00;
end
else if (opcode == 7'b0110011) begin
    aluSrc = 1'b0;
    memtoreg = 1'b0;
    regwrite = 1'b1;
    memread = 1'b0;
    memwrite = 1'b0;
    branch = 1'b0;
    Aluop = 2'b10;
end
else if (opcode == 7'b1100011) begin
    aluSrc = 1'b0;
    memtoreg = 1'bx;
    regwrite = 1'b0;
    memread = 1'b0;
    memwrite = 1'b0;
    branch = 1'b1;
    Aluop = 2'b01;
end
else if (opcode == 7'b0010011) begin
    aluSrc = 1'b1;
    memtoreg = 1'b0;
    regwrite = 1'b1;
    memread = 1'b0;
    memwrite = 1'b0;
    branch = 1'b0;
    Aluop = 2'b00;
end
else begin
    aluSrc = 1'b0;
    memtoreg = 1'b0;
    regwrite = 1'b0;
    memread = 1'b0;
    memwrite = 1'b0;

```

```

        branch = 1'b0;
        Aluop =2'b00;
    end

```

```

    if (stall == 1'b1)
    begin
        aluSrc = 1'b0;
        memtoreg = 1'b0;
        regwrite = 1'b0;
        memread = 1'b0;
        memwrite = 1'b0;
        branch = 1'b0;
        Aluop =2'b00;
    end
end

```

```
endmodule
```

pc.sv

```

/* verilator lint_off EOFNEWLINE */
/* verilator lint_off VARHIDDEN */
module pc(
    input logic [12:0] nxt_pc,
    output logic [12:0] pc_o,
    input logic clk,
    input stalling_signal

);
integer counter;
initial
    begin
        counter = 0;
    end
always @(posedge clk)
begin
    if(stalling_signal)
        begin
            pc_o <= pc_o;
            counter <= counter + 1;
            if(counter == 1)
                begin
                    pc_o <= nxt_pc;
                    counter <= 0;
                end
        end
    else
        begin
            pc_o <= nxt_pc;
        end
end

```

```
end
endmodule
```

IF_ID.sv

```
/*
    Name: Pipeline register between Datapath Fetch and Decode Stage
*/
```

```
module IF_ID (input logic clk, reset, clear, enable,
              input logic [31:0] InstrF, PCF, PCPlus4F,
              output logic [31:0] InstrD, PCD, PCPlus4D);
```

```
always_ff @( posedge clk, posedge reset ) begin
```

```
    if (reset) begin // Asynchronous Clear
```

```
        InstrD <= 0;
```

```
        PCD <= 0;
```

```
        PCPlus4D <= 0;
```

```
    end
```

```
    else if (enable) begin
```

```
        if (clear) begin // Synchrnous Clear
```

```
            InstrD <= 0;
```

```
            PCD <= 0;
```

```
            PCPlus4D <= 0;
```

```
        end
```

```
        else begin
```

```
            InstrD <= InstrF;
```

```
            PCD <= PCF;
```

```
            PCPlus4D <= PCPlus4F;
```

```
        end
```

```
    end
```

```
end
```

```
endmodule
```

ID_IEx.sv

```
/*
    Name: Pipeline register between Datapath Decode and Execution Stage
*/
```

```
module ID_IEx (input logic clk, reset, clear,
               input logic [31:0] RD1D, RD2D, PCD,
               input logic [4:0] Rs1D, Rs2D, RdD,
               input logic [31:0] ImmExtD, PCPlus4D,
               output logic [31:0] RD1E, RD2E, PCE,
               output logic [4:0] Rs1E, Rs2E, RdE,
```

<pre> output logic [31:0] ImmExtE, PCPlus4E); always_ff @(posedge clk, posedge reset) begin if (reset) begin RD1E <= 0; RD2E <= 0; PCE <= 0; Rs1E <= 0; Rs2E <= 0; RdE <= 0; ImmExtE <= 0; PCPlus4E <= 0; end else if (clear) begin RD1E <= 0; RD2E <= 0; PCE <= 0; Rs1E <= 0; Rs2E <= 0; RdE <= 0; ImmExtE <= 0; PCPlus4E <= 0; end else begin RD1E <= RD1D; RD2E <= RD2D; PCE <= PCD; Rs1E <= Rs1D; Rs2E <= Rs2D; RdE <= RdD; ImmExtE <= ImmExtD; PCPlus4E <= PCPlus4D; end end endmodule </pre>	riscv_pip_27.sv <pre> /* Name: RISC V RV32I Description: Implements 27 instructions 5 stage pipeline Hazard Unit is present for handling both control and data hazards */ module riscv_pip_27(input logic clk, reset, output logic [31:0] PCF, </pre>
---	---

```

        input logic [31:0] InstrF,
        output logic MemWriteM,
        output logic [31:0] ALUResultM, WriteDataM,
        input logic [31:0] ReadDataM);

logic ALUSrcAE, RegWriteM, RegWriteW, ZeroE, SignE, PCJalSrcE, PCSrcE;
logic [1:0] ALUSrcBE;
logic StallD, StallF, FlushD, FlushE, ResultSrcE0;
logic [1:0] ResultSrcW;
logic [2:0] ImmSrcD;
logic [3:0] ALUControlE;
logic [31:0] InstrD;
logic [4:0] Rs1D, Rs2D, Rs1E, Rs2E;
logic [4:0] RdE, RdM, RdW;
logic [1:0] ForwardAE, ForwardBE;

controller c(clk, reset, InstrD[6:0], InstrD[14:12], InstrD[30], ZeroE, SignE, FlushE, ResultSrcE0, ResultSrcW, MemWriteM,
PCJalSrcE, PCSrcE, ALUSrcAE, ALUSrcBE, RegWriteM, RegWriteW, ImmSrcD, ALUControlE);

hazardunit h (Rs1D, Rs2D, Rs1E, Rs2E, RdE, RdM, RdW, RegWriteM, RegWriteW, ResultSrcE0, PCSrcE, ForwardAE,
ForwardBE, StallD, StallF, FlushD, FlushE);

datapath dp(clk, reset, ResultSrcW, PCJalSrcE, PCSrcE,ALUSrcAE, ALUSrcBE, RegWriteW, ImmSrcD, ALUControlE, ZeroE,
SignE, PCF, InstrF, InstrD, ALUResultM, WriteDataM, ReadDataM, ForwardAE, ForwardBE, Rs1D, Rs2D, Rs1E, Rs2E, RdE,
RdM, RdW, StallD, StallF, FlushD, FlushE);

endmodule

```

hazardunit.sv

```

/*
    Name: Hazard Unit
    Description: Handles both Data Hazards and Control Hazards
*/
module hazardunit(input logic [4:0] Rs1D, Rs2D, Rs1E, Rs2E,
        input logic [4:0] RdE, RdM, RdW,
        input logic RegWriteM, RegWriteW,
        input logic ResultSrcE0, PCSrcE,
        output logic [1:0] ForwardAE, ForwardBE,
        output logic StallD, StallF, FlushD, FlushE);

// RAW
// Whenever source register (Rs1E, Rs2E) in execution stage matches with the destination register (RdM, RdW)
// of a previous instruction's Memory or WriteBack stage forward the ALUResultM or ResultW
// And also only when RegWrite is asserted

logic lwStall;
always_comb begin
    ForwardAE = 2'b00;
    ForwardBE = 2'b00;

```

```

        if ((Rs1E == RdM) & (RegWriteM) & (Rs1E != 0)) // higher priority - most recent
            ForwardAE = 2'b10; // for forwarding ALU Result in Memory Stage
        else if ((Rs1E == RdW) & (RegWriteW) & (Rs1E != 0))
            ForwardAE = 2'b01; // for forwarding WriteBack Stage Result

        if ((Rs2E == RdM) & (RegWriteM) & (Rs2E != 0))
            ForwardBE = 2'b10; // for forwarding ALU Result in Memory Stage

        else if ((Rs2E == RdW) & (RegWriteW) & (Rs2E != 0))
            ForwardBE = 2'b01; // for forwarding WriteBack Stage Result

        end

// For Load Word Dependency result does not appear until end of Data Memory Access Stage
// if Destination register in EXE stage is equal to source register in decode stage
// stall previous instructions until the the load word is available at the writeback stage
// Introduce One cycle latency for subsequent instructions after load word
// There is two cycle difference between Memory Access and the immediate next instruction

        assign lwStall = (ResultSrcE0 == 1) & ((RdE == Rs1D) | (RdE == Rs2D));
//    assign FlushE = lwStall;
        assign StallF = lwStall;
        assign StallD = lwStall;

// control hazard
// whenever branch has been taken, we flush the following two instructions from decode and execute pip reg
        assign FlushE = lwStall | PCSrcE;
        assign FlushD = PCSrcE;

endmodule

```

datapath.sv

```

/*
    Name: 5 stage pipelined Datapath
    Description: Datapath with 5 stages
    Refer the microarchitecture in the images folder for reference

*/
module datapath(input logic clk, reset,
                input logic [1:0] ResultSrcW,
                input logic PCJalSrcE, PCSrcE, ALUSrcAE,
                input logic [1:0] ALUSrcBE,
                input logic RegWriteW,
                input logic [2:0] ImmSrcD,
                input logic [3:0] ALUControlE,

```

```

        output logic ZeroE,
        output logic SignE,
        output logic [31:0] PCF,
        input logic [31:0] InstrF,
        output logic [31:0] InstrD,
        output logic [31:0] ALUResultM, WriteDataM,
        input logic [31:0] ReadDataM,
        input logic [1:0] ForwardAE, ForwardBE,
        output logic [4:0] Rs1D, Rs2D, Rs1E, Rs2E,
output logic [4:0] RdE, RdM, RdW,
        input logic StallD, StallF, FlushD, FlushE);

logic [31:0] PCD, PCE, ALUResultE, ALUResultW, ReadDataW;
logic [31:0] PCNextF, PCPlus4F, PCPlus4D, PCPlus4E, PCPlus4M, PCPlus4W, PCTargetE, BranJumpTargetE;
logic [31:0] WriteDataE;
logic [31:0] ImmExtD, ImmExtE;
logic [31:0] SrcAEfor, SrcAE, SrcBE, RD1D, RD2D, RD1E, RD2E;
logic [31:0] ResultW;

logic [4:0] RdD; // destination register address

// Fetch Stage

mux2 jal_r(PCTargetE, ALUResultE, PCJalSrcE, BranJumpTargetE);
mux2 pcmux(PCPlus4F, BranJumpTargetE, PCSrcE, PCNextF);
flopennr IF(clk, reset, ~StallF, PCNextF, PCF);
adder pcadd4(PCF, 32'd4, PCPlus4F);

// Instruction Fetch - Decode Pipeline Register

IF_ID pipreg0 (clk, reset, FlushD, ~StallD, InstrF, PCF, PCPlus4F, InstrD, PCD, PCPlus4D);
assign Rs1D = InstrD[19:15];
assign Rs2D = InstrD[24:20];
regfile rf (clk, RegWriteW, Rs1D, Rs2D, RdW, ResultW, RD1D, RD2D);
assign RdD = InstrD[11:7];
extend ext(InstrD[31:7], ImmSrcD, ImmExtD);

// Decode - Execute Pipeline Register

ID_IEx pipreg1 (clk, reset, FlushE, RD1D, RD2D, PCD, Rs1D, Rs2D, RdD, ImmExtD, PCPlus4D, RD1E, RD2E, PCE,
Rs1E, Rs2E, RdE, ImmExtE, PCPlus4E);
mux3 forwardMuxA (RD1E, ResultW, ALUResultM, ForwardAE, SrcAEfor);
mux2 srcamux(SrcAEfor, 32'b0, ALUSrcAE, SrcAE); // for lui
mux3 forwardMuxB (RD2E, ResultW, ALUResultM, ForwardBE, WriteDataE);
mux3 srcbmux(WriteDataE, ImmExtE, PCTargetE, ALUSrcBE, SrcBE);
adder pcaddbranch(PCE, ImmExtE, PCTargetE); // Next PC for jump and branch instructions

```

```
alu alu(SrcAE, SrcBE, ALUControlE, ALUResultE, ZeroE, SignE);

// Execute - Memory Access Pipeline Register
IEx_IMem pipreg2 (clk, reset, ALUResultE, WriteDataE, RdE, PCPlus4E, ALUResultM, WriteDataM, RdM,
PCPlus4M);

// Memory - Register Write Back Stage
IMem_IW pipreg3 (clk, reset, ALUResultM, ReadDataM, RdM, PCPlus4M, ALUResultW, ReadDataW, RdW,
PCPlus4W);
mux3 resultmux( ALUResultW, ReadDataW, PCPlus4W, ResultSrcW, ResultW);

endmodule
```